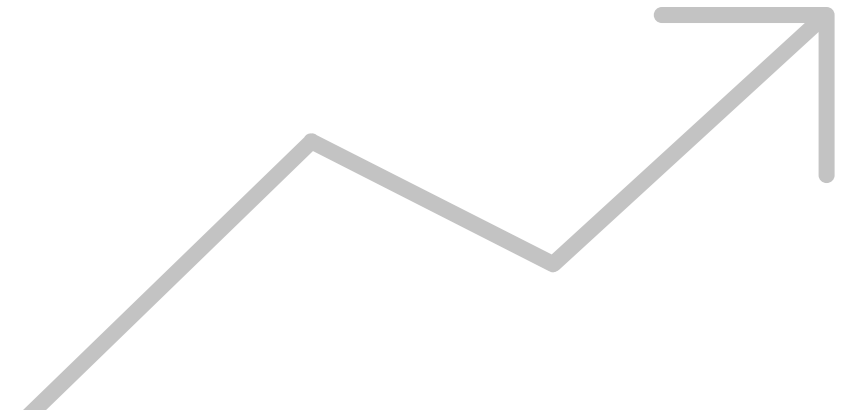


Effiziente Auswertung steuerstatistischer Daten

im FDZ OnSite-Server mithilfe von R

Demonstriert anhand des Business-Tax-Panels (BTP)

Oliver Hauke (Statistisches Bundesamt, IT-Kompetenzzentrum „Analyse und Auswertung“)



Ausgangslage – Daten

Business-Tax-Panel

- NeSt Produkt (2024)
- Großer Datensatz (bspw. 840 GB Stata-Dateien)

2 700+ Merkmale
(Verknüpfung von 6
Unternehmenssteuer-
statistiken sowie dem
Unternehmensregister)

67 Mio. Beobachtungen
(2013-2019)

Datenbereitstellung

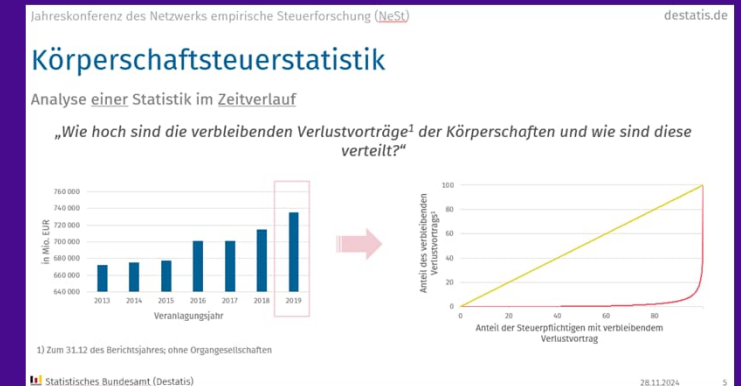
Metadatenreport

Häufigkeit gesamt	Statistik						
	Gewerbe- steuer	Körperschaft- steuer	Umsatzsteuer- Vorankündigung	Statistik über Personengesell- schaften und Gemeinschaften	Umsatzsteuer- -Veranlagung	Statistisches Unternehmens- register	Einnahmen- überschuss- rechnung
2013	3.633.451	1.210.838	3.243.538	1.205.110	6.435.280	3.474.431	3.703.982
2014	3.714.176	1.236.129	3.240.221	1.220.521	6.446.620	3.550.775	3.720.318
2015	3.819.938	1.264.717	3.255.537	1.234.537	6.535.948	3.634.643	4.230.871
2016	3.887.556	1.293.431	3.266.429	1.247.699	6.550.960	3.674.354	4.581.982
2017	3.981.853	1.316.520	3.266.806	1.220.067	6.664.535	3.708.949	5.685.661
2018	4.050.674	1.350.837	3.279.136	1.284.121	6.831.036	3.736.550	6.310.028
2019	4.099.690	1.388.679	3.288.306	1.294.898	6.972.252	3.723.169	6.501.292

In SAS ca. 54 min Laufzeit

https://www.forschungsdatenzentrum.de/sites/default/files/btp_2013-2019_on-site_mdr-prod.pdf

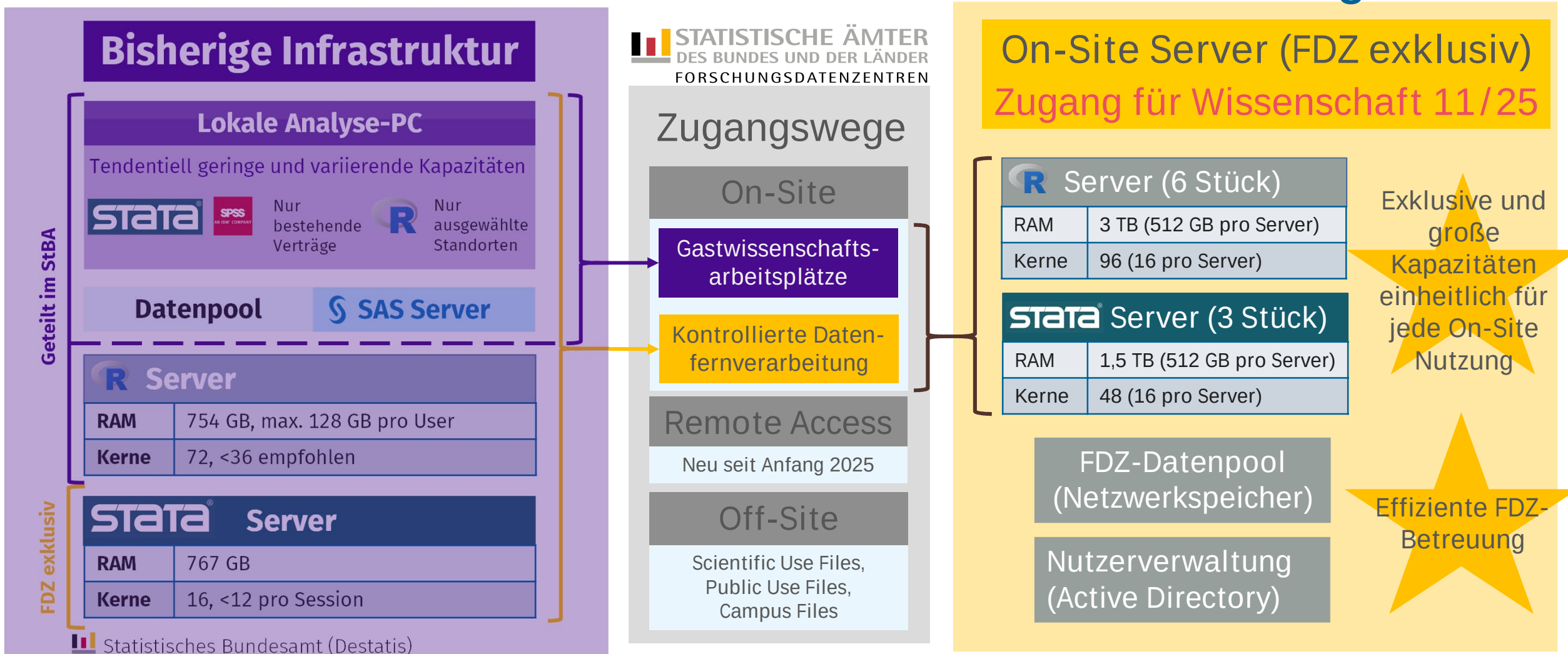
Wissenschaftliche Auswertung



In SAS ca. 2h Laufzeit

https://www.bundesfinanzministerium.de/Content/DE/Standardartikel/Themen/Steuern/Steuerliche_Themengebiete/NeSt/nest-jahreskonferenz-programm.html

FDZ-Infrastruktur und deren Weiterentwicklung





R bietet die Mittel für Effizienzgewinne!

Technischer Ansatz



{Parquet} Speicherformat

- Effizient lesbar und schreibbar
- Spaltenbasiert
- Standardmäßig komprimiert
- Ermöglicht aufgeteilte Speicherung



{arrow} R-Package

- Nutzung von Daten außerhalb des Arbeitsspeichers
- Lazy Computation
- Parallelisierung
- Gewohnte R-Syntax



DuckDB ergänzt

Features von {arrow}.



{data.table}
kann in-memory
effizienter sein.



Weiterführende Literatur

- <https://www.forschungsdatenzentrum.de/de/steuern/btp>
- <https://parquet.apache.org/>
- <https://arrow.apache.org/>
- <https://arrow.apache.org/docs/r/>
- <https://duckdb.org/>
- <https://r4ds.hadley.nz/arrow#partitioning>
- <https://arrow.apache.org/cookbook/r>
- <https://github.com/thisisnic/awesome-arrow-r>
- <https://confluence.prod.stba.in.bund.de/x/qY7FAg>
- <https://arrow.apache.org/cookbook/r/manipulating-data---tables.html#use-dplyr-verbs-in-arrow>
- <https://arrow.apache.org/docs/r/articles/dataset.html#batch-processing-experimental>

1 Vorbereitung

1.1 Pakete laden

1.2 Verbindung zu Daten herstellen

2 Datenübersicht

2.1 Fallzahlen

2.2 Numerische Kennzahlen

3 Auswertung

3.1 Lorenzkurve (Verlustvorträge der Körperschaften)

3.2 Logistische Regression (Einflüsse auf positive Verlustvorträge)

Effizientes Programmieren mit R und technische Weiterentwicklung des FDZ

Code ▼

Anhand des Business-Tax-Panel (BTP)

Oliver Hauke (C31)

11.11.2025

1 Vorbereitung

1.1 Pakete laden

Hide

```
library(arrow)      # Effizienz, R-nativ
library(duckdb)     # Effizienz, SQL-artig
library(dplyr)      # Datenverarbeitung
library(tidyr)      # Datenverarbeitung
library(ggplot2)    # Plots
library(here)       # Navigation im Dateisystem
library(lobstr)     # Speichermessung

arrow::set_cpu_count(16)
proj_path <- file.path("~/auswertung/btp")
```

1.2 Verbindung zu Daten herstellen

Hide

```
fs::dir_info("~/auswertung/btp/daten/") |>
  mutate(filename = basename(path)) |>
  select(filename, size) |>
  filter(grepl("btp", filename)) |>
  arrange(desc(filename))
```

```
#> # A tibble: 28 × 2
#>   filename                size
#>   <chr>                  <fs::bytes>
#> 1 btp_kdfv_2019_v1.parquet    2.76G
#> 2 btp_kdfv_2019_v1.csv       26.38G
#> 3 btp_kdfv_2019_v0.zip        4.55G
#> 4 btp_kdfv_2019_v0.dta      100.44G
#> 5 btp_kdfv_2018_v1.parquet    2.78G
#> 6 btp_kdfv_2018_v1.csv       25.9G
#> 7 btp_kdfv_2018_v0.zip        4.21G
#> 8 btp_kdfv_2018_v0.dta       95.72G
#> 9 btp_kdfv_2017_v1.parquet    2.73G
#> 10 btp_kdfv_2017_v1.csv      24.75G
#> # i 18 more rows
```

Hide

```
system.time({
  pq_files <- list.files("~/auswertung/btp/daten/", pattern= ".parquet", full.names = T
RUE)
  da <- open_dataset(pq_files, partitioning = "Jahr")
})
obj_size(da)
```

```
#>   user system elapsed
#>  0.171   0.074   0.576
#> 263.67 kB
```

Hide

```
system.time({
  da |>
    dim() |>
    print()
})
```

```
#> [1] 66993043    2265
#>   user system elapsed
#>  1.010   0.608   1.667
```

Hide

2 Datenübersicht

```
system.time({
  da |>
    select(3 + 1:20) |>
    head(100) |>
    collect() |>
    str()
})
```

```
#> tibble [100 × 20] (S3: tbl_df/tbl/data.frame)
#> $ verk_qual: int [1:100] 1 1 1 1 1 1 1 1 2 1 ...
#> $ g_ef4 : int [1:100] 3 3 3 3 NA 3 3 3 3 3 ...
#> $ g_fef14 : int [1:100] NA 1 NA NA NA NA NA NA NA ...
#> $ g_fef17 : int [1:100] 3 4 3 3 NA 4 3 3 4 1 ...
#> $ g_fef19 : int [1:100] NA NA NA NA NA NA NA NA NA ...
#> $ g_fef20 : chr [1:100] "F" "K" "K" "K" ...
#> $ g_fef21 : int [1:100] NA NA NA NA NA NA NA NA 3 NA ...
#> $ g_fef25 : logi [1:100] NA NA NA NA NA NA ...
#> $ g_fef26 : int [1:100] NA NA NA NA NA NA NA NA NA ...
#> $ g_fef27 : int [1:100] NA NA NA NA NA NA NA NA NA ...
#> $ g_fef28 : int [1:100] 4 4 4 4 NA 4 4 4 4 4 ...
#> $ g_k2110 : int [1:100] 175017 2579583 59005 165186 NA 1596614 20704 6335 -66530 5
000 ...
#> $ g_k2114 : int [1:100] 0 0 0 0 NA 0 0 0 0 0 ...
#> $ g_k2116 : int [1:100] 0 0 0 0 NA 0 0 0 0 0 ...
#> $ g_k2117 : int [1:100] 0 0 0 0 NA 0 0 0 0 0 ...
#> $ g_k2119 : int [1:100] 0 0 0 0 NA 0 0 0 0 0 ...
#> $ g_k2121 : int [1:100] NA NA NA NA NA NA NA NA NA ...
#> $ g_k2122 : int [1:100] 0 0 0 0 NA 0 0 0 0 0 ...
#> $ g_k2126 : int [1:100] 0 0 0 0 NA 0 0 0 0 0 ...
#> $ g_k2127 : int [1:100] 0 0 0 0 NA 0 0 0 0 0 ...
#> user system elapsed
#> 2.015 0.317 0.726
```

2.1 Fallzahlen

2.1.1 Fallzahlen pro Jahr

Hide

```
system.time({
  fallzahlen_jahr <- da |>
    group_by(Jahr) |>
    count() |>
    ungroup() |>
    collect()
})
fallzahlen_jahr
```



```
#>   user  system elapsed
#> 2.123   0.011   1.130
#> # A tibble: 7 × 2
#>   Jahr      n
#>   <int>   <int>
#> 1  2014 8759322
#> 2  2015 9051437
#> 3  2013 8689793
#> 4  2016 9260593
#> 5  2018 10491366
#> 6  2017 9989806
#> 7  2019 10750726
```

2.1.2 Fallzahlen pro Jahr und Statistik

Vorgehen

Hide

```
system.time({
  da|>
    select(Jahr, verk) |>
    head() |>
    collect()
})
```

```
#>   user  system elapsed
#> 0.754   0.054   0.498
```

Hide

```
system.time({

  fallzahlen_stat <- da|>

    select(Jahr, verk) |>

    mutate(
      e = str_detect(verk, "e"),
      g = str_detect(verk, "g"),
      k = str_detect(verk, "k"),
      p = str_detect(verk, "p"),
      r = str_detect(verk, "r"),
      u = str_detect(verk, "u"),
      v = str_detect(verk, "v")) |>

    group_by(Jahr) |>

    summarize(across(where(is.logical), sum)) |>

    arrange(Jahr) |>

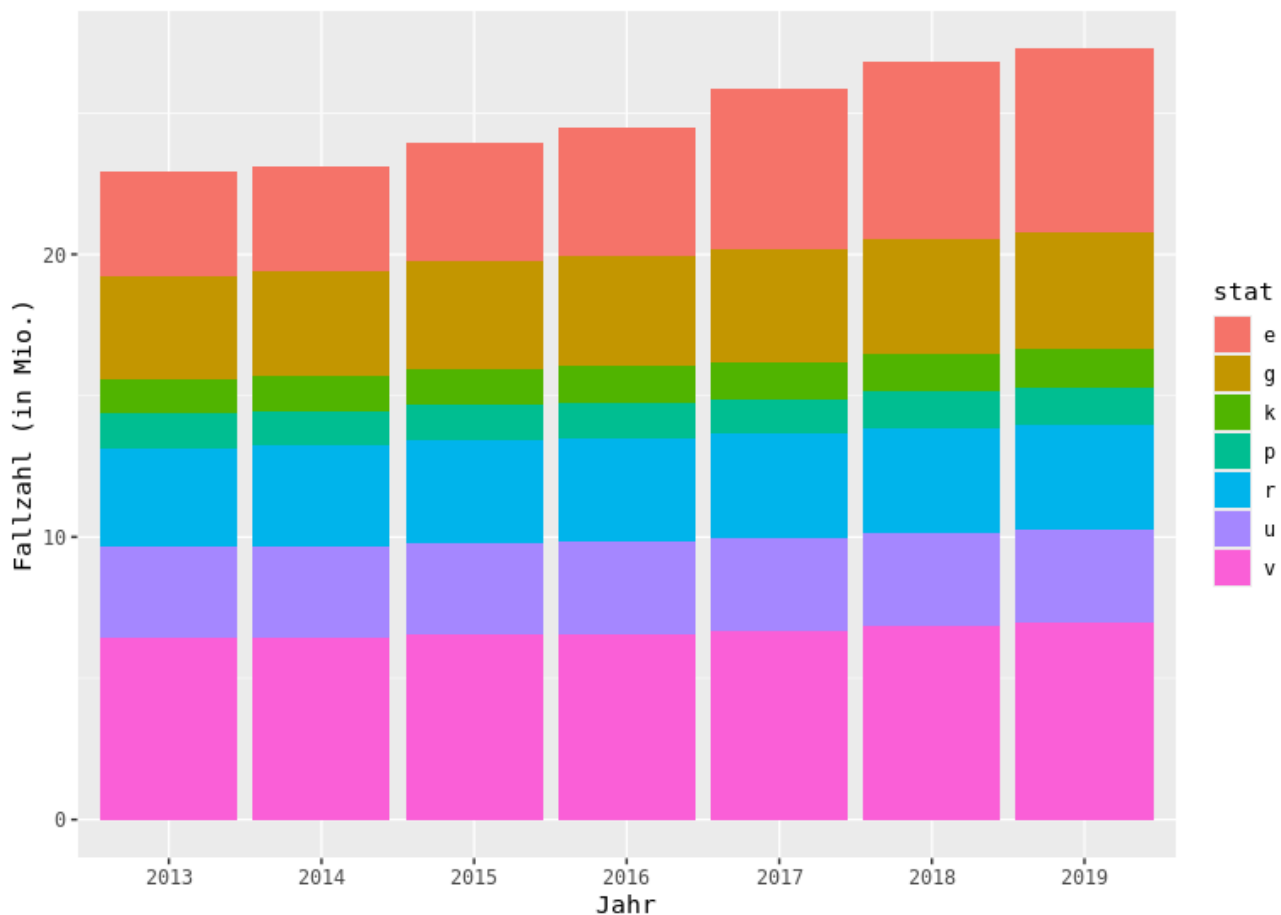
    collect()
})
fallzahlen_stat
```

```
#>   user  system elapsed
#> 27.121   0.000   2.296
#> # A tibble: 7 × 8
#>   Jahr      e      g      k      p      r      u      v
#>   <int>   <int>   <int>   <int>   <int>   <int>   <int>   <int>
#> 1  2013 3703982 3633451 1210838 1205110 3474431 3243538 6435280
#> 2  2014 3720318 3714176 1236129 1220521 3550775 3240221 6446620
#> 3  2015 4230871 3819938 1264717 1234537 3634643 3255537 6535948
#> 4  2016 4581982 3887556 1293431 1247699 3674354 3266429 6550960
#> 5  2017 5685661 3981853 1316520 1220067 3708949 3266806 6664535
#> 6  2018 6310028 4050674 1350837 1284121 3736550 3279136 6831036
#> 7  2019 6501292 4099690 1388679 1294898 3723169 3288306 6972252
```

Hide

```
fallzahlen_stat |>
  pivot_longer(-Jahr, names_to="stat", values_to = "n") |>
  mutate(Jahr = factor(Jahr), Fallzahl = n / 1e6) |>
  ggplot(aes(x = Jahr, y = Fallzahl, fill = stat)) +
  geom_bar(stat="identity") +
  ylab("Fallzahl (in Mio.)") +
  ggtitle("BTP - Fallzahlen nach Jahr und Statistik")
```

BTP - Fallzahlen nach Jahr und Statistik



2.2 Numerische Kennzahlen

Hide

```

system.time({
  overview <- function(da){

    select(da, where(is.numeric)) |>

    summarize(
      across(.fns= ~ sum(!is.na(.x)), .names = "{.col}-obs"),
      across(.fns= ~ sum( is.na(.x)), .names = "{.col}-nna"),
      across(.fns= ~ quantile(.x, 0.1, na.rm = TRUE), .names = "{.col}-q10"),
      across(.fns= ~ mean(.x, na.rm = TRUE), .names = "{.col}-mean"),
      across(.fns= ~ median(.x, na.rm = TRUE), .names = "{.col}-median"),
      across(.fns= ~ quantile(.x, 0.9, na.rm = TRUE), .names = "{.col}-q90"),
      across(.fns= ~ sd(.x, na.rm = TRUE), .names = "{.col}-sd")
    )
  }

# Uebersicht ausrechnen
descr <- da |>
  filter(str_detect(verk, "p")) |>           # Personengesellschaften
  select(p_ef35:p_ef37, p_ef58, p_ef59, p_ef61, p_ef62) |>
  overview() |>
  collect()

})

# schoenere Darstellung
descr |>
  mutate(across(everything(), as.numeric)) |>
  pivot_longer(cols = everything(), names_to = "key", values_to = "value") |>
  separate(key, into = c("Variable", "metrik"), sep = "-") |>
  pivot_wider(names_from = metrik, values_from = value) |>
  mutate(across(where(is.numeric), as.integer))

```

```

#>   user  system elapsed
#> 10.042   0.428   1.736
#> # A tibble: 6 × 8
#>   Variable    obs    nna    q10   mean median    q90    sd
#>   <chr>      <int> <int> <int> <int> <int> <int> <int>
#> 1 p_ef35    13860 8693093 -324870 6279143  80120 7996924 73228254
#> 2 p_ef36    655352 8051601  -1717  285271 126772  625043  1391716
#> 3 p_ef37    676633 8030320     0    79330     67  11339 12171125
#> 4 p_ef58   3839496 4867457  -4627   17445   3488   39450   484638
#> 5 p_ef59    37732 8669221 -11350   93753   5498  130806  1342766
#> 6 p_ef62   8686088  20865  -5299  144568   6444  195169  7739366

```

3 Auswertung

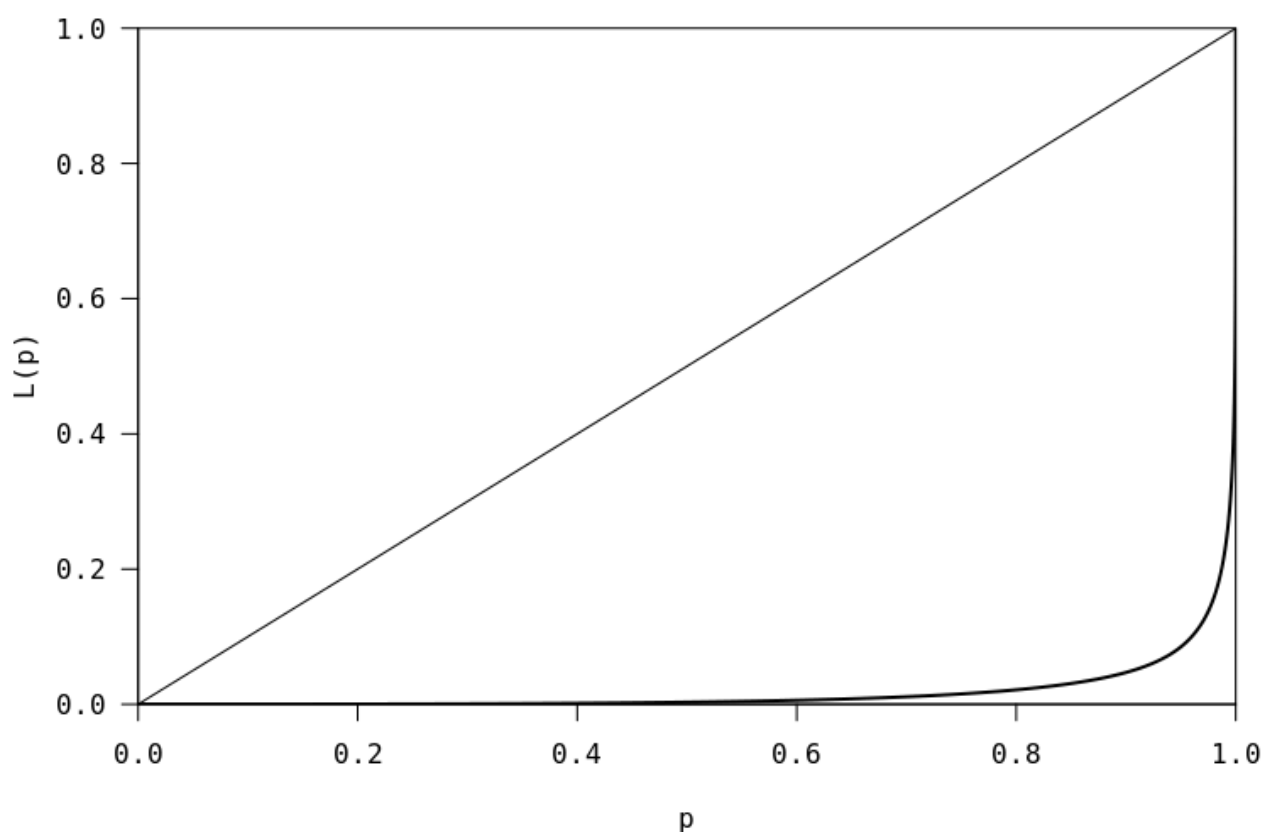
3.1 Lorenzkurve (Verlustvorträge der Körperschaften)

[Hide](#)

```
system.time({  
  var_lc <- da |>  
    filter(str_detect(verk, "k"), Jahr == 2019) |> # nur Koerperschaftsteuerstat. in 20  
19  
    pull(k_k65270)                                # Verlustvortraege  
  
  lc <- ineq::Lc(var_lc)  
  
  plot(lc)  
})
```

```
#>   user  system elapsed  
#>  1.874    0.056    1.280
```

Lorenz curve



3.2 Logistische Regression (Einflüsse auf positive Verlustvorträge)

[Hide](#)

```

system.time({
  da_clean <- da |> filter(str_detect(verk, "k") & str_detect(verk, "u")) |>
    select(
      ID, Jahr,
      k_k65270, # Verlustvorträge
      k_k65823, # Gesamtbetrag der Einkünfte
      k_c15018, # Summe der Umsätze/Löhne/Gehälter
      k_ef20,   # WZ Generierung
      k_k65172, # Spende

      urs_we_tp_stichtag, # Tätige Personen
      urs_rt_gruppen_kennz # Statuskennzeichen, = 3/6 heißt "auslandskontrolliert"
    ) |>

  compute() |>

  mutate(

    # Positiver Verlustvortrag vorhanden?
    vl_dummy = ifelse(k_k65270 > 0 & !is.na(k_k65270), 1, 0),

    # International taetig?
    ifats_dummy = ifelse(urs_rt_gruppen_kennz %in% c(3,6), 1, 0),

    # Wirtschaftszweig
    key = as.numeric(substr(k_ef20,1,2)),
    wz = case_when(
      key %in% 1:3 ~ "A",
      key %in% 5:9 ~ "B",
      key %in% 10:33 ~ "C",
      key %in% 35 ~ "D",
      key %in% 36:39 ~ "E",
      key %in% 41:43 ~ "F",
      key %in% 45:47 ~ "G",
      key %in% 49:53 ~ "H",
      key %in% 55:56 ~ "I",
      key %in% 58:63 ~ "J",
      key %in% 64:66 ~ "K",
      key %in% 68 ~ "L",
      key %in% 69:75 ~ "M",
      key %in% 77:82 ~ "N",
      key %in% 84 ~ "O",
      key %in% 85 ~ "P",
      key %in% 86:88 ~ "Q",
      key %in% 90:93 ~ "R",
      key %in% 99 ~ "U",
      .default = NA)

    , .keep="unused"

  ) |> select(-key) |>
  mutate(across(where(is.numeric), as.numeric))

mod <- da_clean |>
  # Shift der erklärenden Variablen um ein Jahr

```

```

left_join(
  mutate(da_clean, Jahr = Jahr + 1),
  by = c("ID", "Jahr"), suffix = c("", "_t1")
) |>
select(
  vl_dummy, Jahr,
  k_k65823_t1, k_c15018_t1, k_k65172_t1,
  ifats_dummy_t1, urs_we_tp_stichtag_t1, wz) %>%

# Modell
{glm("vl_dummy ~ .", data = ., family = binomial)}
})
summary(mod)

```

```

#>   user  system elapsed
#> 25.593 15.265 11.959
#>
#> Call:
#> glm(formula = "vl_dummy ~ .", family = binomial, data = .)
#>
#> Coefficients:
#>               Estimate Std. Error z value Pr(>|z|)
#> (Intercept)      7.077e+01  2.787e+00  25.396 < 2e-16 ***
#> Jahr             -3.549e-02  1.382e-03 -25.681 < 2e-16 ***
#> k_k65823_t1      -9.556e-08  1.812e-09 -52.746 < 2e-16 ***
#> k_c15018_t1       4.283e-11  1.619e-11   2.646 0.008138 **
#> k_k65172_t1       8.668e-07  5.192e-08 16.697 < 2e-16 ***
#> ifats_dummy_t1    2.920e-01  1.252e-02 23.326 < 2e-16 ***
#> urs_we_tp_stichtag_t1 -3.369e-05  6.893e-06 -4.887 1.02e-06 ***
#> wzD              -3.689e-01  2.965e-02 -12.441 < 2e-16 ***
#> wzE              -2.790e-01  2.626e-02 -10.623 < 2e-16 ***
#> wzF              -1.129e-01  7.936e-03 -14.229 < 2e-16 ***
#> wzG               7.684e-03  7.257e-03   1.059 0.289718
#> wzH              -5.308e-02  1.409e-02  -3.768 0.000164 ***
#> wzI               4.306e-01  1.375e-02 31.303 < 2e-16 ***
#> wzJ               2.622e-03  1.070e-02   0.245 0.806468
#> wzK              -2.327e-01  2.012e-02 -11.566 < 2e-16 ***
#> wzL               4.850e-01  1.291e-02 37.575 < 2e-16 ***
#> wzM              -2.028e-01  8.579e-03 -23.639 < 2e-16 ***
#> wzN              -5.393e-02  1.127e-02  -4.785 1.71e-06 ***
#> wzP               3.743e-01  2.737e-02 13.675 < 2e-16 ***
#> wzQ              -7.446e-02  2.848e-02  -2.614 0.008947 **
#> wzR               5.465e-01  1.864e-02 29.316 < 2e-16 ***
#> wzU              -8.841e-01  5.438e-01  -1.626 0.103970
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
#>
#> (Dispersion parameter for binomial family taken to be 1)
#>
#>   Null deviance: 1066800  on 868816  degrees of freedom
#> Residual deviance: 1049118  on 868795  degrees of freedom
#> (3320918 observations deleted due to missingness)
#> AIC: 1049162
#>
#> Number of Fisher Scoring iterations: 8

```